Character-Level Language Model from Scratch

A step-by-step implementation of a simple neural language model trained on *Frankenstein* by Mary Shelley.

We build everything from scratch using only NumPy — no PyTorch, no TensorFlow.

Cell 1: Imports

Import the libraries we'll need throughout the notebook.

In [1]:
```
# Standard libraries
import math
import random
import numpy as np
```

Cell 2: Load the Raw Text

Read the full text of *Frankenstein* from a local file. We print the total length and a preview of the first 500 characters to confirm it loaded correctly.

In [2]:
```
# Load the raw text file (download from Project Gutenberg if you don't have
with open("frankenstein.txt", "r", encoding="utf-8") as f:
    text = f.read()

print("Length:", len(text))
print(text[:500])
```

```
Length: 438806
The Project Gutenberg eBook of Frankenstein; or, the modern prometheus

This ebook is for the use of anyone anywhere in the United States and
most other parts of the world at no cost and with almost no restrictions
whatsoever. You may copy it, give it away or re-use it under the terms
of the Project Gutenberg License included with this ebook or online
at www.gutenberg.org. If you are not located in the United States,
you will have to check the laws of the country where you are located
before
```

Cell 3: Clean the Text (Remove Gutenberg Boilerplate)

Project Gutenberg texts include legal/licensing text at the start and end. We find the `*** START OF` and `*** END OF` markers and slice out only the actual story.

In [3]:

```
start_marker = "*** START OF"  # Marker indicating where the story begins
end_marker = "*** END OF"      # Marker indicating where the story ends

start = text.find(start_marker)  # Find start index of marker
end = text.find(end_marker)      # Find end index of marker

# If both markers exist, slice the text to only include the story
if start != -1 and end != -1:
    text = text[start + len(start_marker):end].strip()  # +len(...) skips th

# Print cleaned text length and first 500 characters
print("Cleaned text length:", len(text))
print(text[:500])
```

```
Cleaned text length: 419409
THE PROJECT GUTENBERG EBOOK FRANKENSTEIN; OR, THE MODERN PROMETHEUS ***

Frankenstein;

or, the Modern Prometheus

by Mary Wollstonecraft (Godwin) Shelley


 CONTENTS

 Letter 1
 Letter 2
 Letter 3
 Letter 4
 Chapter 1
 Chapter 2
 Chapter 3
 Chapter 4
 Chapter 5
 Chapter 6
 Chapter 7
 Chapter 8
 Chapter 9
 Chapter 10
 Chapter 11
 Chapter 12
 Chapter 13
 Chapter 14
 Chapter 15
 Chapter 16
 Chapter 17
 Chapter 18
 Chapter 19
 Chapter 20
 Chapter 21
 Chapter 22
 Chapter 23
 Chapter 24




Letter 1
```

Cell 4: Build the Character Vocabulary

We identify all unique characters in the text and create two mappings:

- `stoi` (string-to-integer): maps each character to a unique integer ID
- `itos` (integer-to-string): maps each integer ID back to its character

These mappings are how we convert between human-readable text and numeric tokens the model can process.

In [4]:
```
# Identify all unique characters in the text
# 'set(text)' returns only unique characters
# 'sorted(...)' ensures a consistent, reproducible order
chars = sorted(list(set(text)))

# Total number of unique characters (this is our vocabulary size)
vocab_size = len(chars)

# Mapping from character -> integer index
stoi = {ch: i for i, ch in enumerate(chars)}

# Mapping from integer index -> character
itos = {i: ch for i, ch in enumerate(chars)}

# Inspect vocabulary
print("Vocab size:", vocab_size)
print("All unique characters:", chars)
```

```
Vocab size: 84
All unique characters: ['\n', ' ', '!', '(', ')', '*', ',', '-', '.', '0',
'1', '2', '3', '4', '5', '6', '7', '8', '9', ':', ';', '?', 'A', 'B', 'C',
'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'R', 'S',
'T', 'U', 'V', 'W', 'Y', '[', ']', '_', 'a', 'b', 'c', 'd', 'e', 'f', 'g',
'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v',
'w', 'x', 'y', 'z', 'æ', 'è', 'é', 'ê', 'ô', '—', '‘', '’', '“', '”']
```

Cell 5: Encode / Decode Helper Functions

Define two helper functions:

- `encode(s)` : converts a string into a list of integer token IDs
- `decode(l)` : converts a list of integer token IDs back into a string

These are the tokenizer for our character-level model.

In [5]:
```
def encode(s):
    """Convert a string into a list of integer token IDs"""
    return [stoi[c] for c in s]

def decode(l):
    """Convert a list of token IDs back into a string"""
    return ''.join([itos[i] for i in l])
```

Cell 6: Print the Character-to-ID Mappings

Visualize both mappings so we can see exactly how characters are encoded.

In [6]:
```python
# Print stoi mapping (character -> ID)
print("stoi mapping:")
print(", ".join([f"{repr(ch)}:{i}" for ch, i in stoi.items()]))

print()

# Print itos mapping (ID -> character)
print("itos mapping:")
print(", ".join([f"{repr(ch)}:{i}" for ch, i in itos.items()]))
```

```
stoi mapping:
'\n':0, ' ':1, '!':2, '(':3, ')':4, '*':5, ',':6, '-':7, '.':8, '0':9, '1':1
0, '2':11, '3':12, '4':13, '5':14, '6':15, '7':16, '8':17, '9':18, ':':19,
';':20, '?':21, 'A':22, 'B':23, 'C':24, 'D':25, 'E':26, 'F':27, 'G':28, 'H':
29, 'I':30, 'J':31, 'K':32, 'L':33, 'M':34, 'N':35, 'O':36, 'P':37, 'R':38,
'S':39, 'T':40, 'U':41, 'V':42, 'W':43, 'Y':44, '[':45, ']':46, '_':47, 'a':
48, 'b':49, 'c':50, 'd':51, 'e':52, 'f':53, 'g':54, 'h':55, 'i':56, 'j':57,
'k':58, 'l':59, 'm':60, 'n':61, 'o':62, 'p':63, 'q':64, 'r':65, 's':66, 't':
67, 'u':68, 'v':69, 'w':70, 'x':71, 'y':72, 'z':73, 'æ':74, 'è':75, 'é':76,
'ê':77, 'ô':78, '—':79, '‘':80, '’':81, '“':82, '”':83

itos mapping:
0:
, 1: , 2:!, 3:(, 4:), 5:*, 6:,, 7:-, 8:., 9:0, 10:1, 11:2, 12:3, 13:4, 14:5,
15:6, 16:7, 17:8, 18:9, 19::, 20:;, 21:?, 22:A, 23:B, 24:C, 25:D, 26:E, 27:
F, 28:G, 29:H, 30:I, 31:J, 32:K, 33:L, 34:M, 35:N, 36:O, 37:P, 38:R, 39:S, 4
0:T, 41:U, 42:V, 43:W, 44:Y, 45:[, 46:], 47:_, 48:a, 49:b, 50:c, 51:d, 52:e,
53:f, 54:g, 55:h, 56:i, 57:j, 58:k, 59:l, 60:m, 61:n, 62:o, 63:p, 64:q, 65:
r, 66:s, 67:t, 68:u, 69:v, 70:w, 71:x, 72:y, 73:z, 74:æ, 75:è, 76:é, 77:ê, 7
8:ô, 79:—, 80:‘, 81:’, 82:“, 83:”
```

Cell 7: Encode the Entire Text

Convert the full cleaned text into a list of integer token IDs. Then verify by decoding a small sample back to text.

In [7]:
```python
# Encode the entire text into a list of integers
encoded_text = encode(text)

# Sanity check: print first 20 token IDs
print("First 20 token IDs:", encoded_text[:20])

# Decode them back to verify correctness
decoded_sample = decode(encoded_text[:20])
print("Decoded text:", repr(decoded_sample))
```

```
First 20 token IDs: [40, 29, 26, 1, 37, 38, 36, 31, 26, 24, 40, 1, 28, 41, 4
0, 26, 35, 23, 26, 38]
Decoded text: 'THE PROJECT GUTENBER'
```

Cell 8: Visualize Token ID ↔ Character Pairs

For the first 20 tokens, print each ID alongside its decoded character. This helps build intuition for how the encoding works.

In [8]:
```
# Create a list of tuples: (ID, character)
id_char_pairs = [(i, itos[i]) for i in encoded_text[:20]]

# Print neatly
for idx, char in id_char_pairs:
    print(f"ID {idx:2} -> {repr(char)}")

print()
# Also print as a compact inline view
print(" | ".join([f"{i}:{repr(itos[i])}" for i in encoded_text[:20]]))
```

```
ID 40 -> 'T'
ID 29 -> 'H'
ID 26 -> 'E'
ID  1 -> ' '
ID 37 -> 'P'
ID 38 -> 'R'
ID 36 -> 'O'
ID 31 -> 'J'
ID 26 -> 'E'
ID 24 -> 'C'
ID 40 -> 'T'
ID  1 -> ' '
ID 28 -> 'G'
ID 41 -> 'U'
ID 40 -> 'T'
ID 26 -> 'E'
ID 35 -> 'N'
ID 23 -> 'B'
ID 26 -> 'E'
ID 38 -> 'R'

40:'T' | 29:'H' | 26:'E' | 1:' ' | 37:'P' | 38:'R' | 36:'O' | 31:'J' | 2
6:'E' | 24:'C' | 40:'T' | 1:' ' | 28:'G' | 41:'U' | 40:'T' | 26:'E' | 35:'N'
| 23:'B' | 26:'E' | 38:'R'
```

Cell 9: Create Input-Target Sequences

This is where we create training data for the model.

Concept: Given a window of `block_size` characters (the "context"), the model's job is to predict the very next character (the "target").

We slide this window across the entire text to produce thousands of (input, target) pairs.

Example with block_size=8:

- Input: `"Chapter "` → Target: `"1"`
- Input: `"hapter 1"` → Target: `"\n"`

In [9]:
```
# block_size = number of previous characters the model sees (context window)
block_size = 8  # small context window for a toy model

# Prepare empty lists to store sequences
x_manual = []  # input sequences (lists of integers)
y_manual = []  # target characters (next character after input)

# Loop over the encoded text to create sequences
# We stop at len(encoded_text) - block_size to ensure each input has exactly
for i in range(len(encoded_text) - block_size):
    # Slice the encoded text from i to i+block_size → the "context" the mode
    input_seq = encoded_text[i:i+block_size]
    x_manual.append(input_seq)

    # The target is the very next character (token ID) after the input seque
    target = encoded_text[i + block_size]
    y_manual.append(target)

print(f"Total training examples: {len(x_manual)}")
```

```
Total training examples: 419401
```

Cell 10: Inspect Training Examples

Print the first 5 input-target pairs to verify our data preparation is correct.

In [10]:
```
for i in range(5):
    print(f"Example {i+1}")
    print("Input IDs:     ", x_manual[i])          # token IDs of input
    print("Decoded Input: ", decode(x_manual[i]))  # convert IDs back to cha
    print("Target ID:     ", y_manual[i])          # token ID of target
    print("Decoded Target:", itos[y_manual[i]])    # convert ID to character
    print("---")
```

```
Example 1
Input IDs:      [40, 29, 26, 1, 37, 38, 36, 31]
Decoded Input:  THE PROJ
Target ID:      26
Decoded Target: E
---
Example 2
Input IDs:      [29, 26, 1, 37, 38, 36, 31, 26]
Decoded Input:  HE PROJE
Target ID:      24
Decoded Target: C
---
Example 3
Input IDs:      [26, 1, 37, 38, 36, 31, 26, 24]
Decoded Input:  E PROJEC
Target ID:      40
Decoded Target: T
---
Example 4
Input IDs:      [1, 37, 38, 36, 31, 26, 24, 40]
Decoded Input:   PROJECT
Target ID:      1
Decoded Target:
---
Example 5
Input IDs:      [37, 38, 36, 31, 26, 24, 40, 1]
Decoded Input:  PROJECT
Target ID:      28
Decoded Target: G
---
```

Cell 11: Initialize Model Parameters

We define a simple feedforward neural network with:

1. **Embedding layer** (`W_embed`): Converts each token ID into a dense vector of size `embedding_dim`
2. **Hidden layer** (`W1` , `b1`): Takes the flattened concatenation of all embeddings and maps to `hidden_dim` neurons with tanh activation
3. **Output layer** (`W2` , `b2`): Maps hidden state to logits over the vocabulary (one score per character)

All weights are initialized with small random values; biases start at zero.

In [11]:
```
# Hyperparameters for our toy model
vocab_size = len(chars)     # number of unique characters
embedding_dim = 16          # size of each character embedding vector
hidden_dim = 32             # size of hidden layer
block_size = 8              # context length (number of previous characters)

# Embedding matrix: maps token IDs -> dense vectors
# Shape: vocab_size x embedding_dim
```

```
W_embed = np.random.randn(vocab_size, embedding_dim) * 0.01

# Hidden layer weights: flattened embeddings -> hidden_dim
# Input size = block_size * embedding_dim (all embeddings concatenated)
W1 = np.random.randn(block_size * embedding_dim, hidden_dim) * 0.01
b1 = np.zeros(hidden_dim)  # bias for hidden layer

# Output layer: hidden_dim -> vocab_size logits
W2 = np.random.randn(hidden_dim, vocab_size) * 0.01
b2 = np.zeros(vocab_size)  # bias for output layer

# Sanity check: print shapes
print("Embedding matrix shape:", W_embed.shape)
print("Hidden layer weight shape:", W1.shape)
print("Output layer weight shape:", W2.shape)
```

```
Embedding matrix shape: (84, 16)
Hidden layer weight shape: (128, 32)
Output layer weight shape: (32, 84)
```

Cell 12: Softmax and Forward Pass Functions

Softmax: Converts raw logits into a valid probability distribution (all positive, sums to 1). We subtract the max for numerical stability to prevent overflow in `exp()`.

Forward pass: The full pipeline from input token IDs to output probabilities:

1. Look up embeddings for each token
2. Flatten all embeddings into one long vector
3. Pass through hidden layer with tanh activation
4. Compute output logits
5. Apply softmax to get probabilities

In [12]:
```
def softmax(x):
    """Compute softmax probabilities for a 1D array.
    Subtracting max(x) prevents numerical overflow."""
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum()


def forward(x_seq):
    """
    Forward pass for a single input sequence.

    Args:
        x_seq: list of token IDs (length = block_size)

    Returns:
        probs: probability distribution over vocab for the next character
    """
    # Step 1: Lookup embeddings for each token ID
    embeds = W_embed[x_seq]        # shape: block_size x embedding_dim
```

```
    # Step 2: Flatten embeddings into a single vector
    h_input = embeds.flatten()     # shape: block_size * embedding_dim

    # Step 3: Hidden layer with tanh activation
    h = np.tanh(h_input @ W1 + b1) # shape: hidden_dim

    # Step 4: Output layer -> logits (one score per vocab character)
    logits = h @ W2 + b2           # shape: vocab_size

    # Step 5: Softmax -> probabilities
    probs = softmax(logits)

    return probs
```

Cell 13: Test the Forward Pass (Before Training)

Run the model on the first training example to see what it predicts *before* any training. Since weights are random, the prediction will be essentially random too.

In [13]:
```
# Take first input sequence and its target
x0 = x_manual[0]
y0 = y_manual[0]

# Forward pass: get predicted probability distribution
probs = forward(x0)

# Predicted character = highest probability
pred_id = np.argmax(probs)

# Print input, target, and prediction
print("Input sequence (decoded):", decode(x0))
print("Target character:", itos[y0])
print("Predicted character (before training):", itos[pred_id])

# Show probabilities of top 5 characters
top5 = np.argsort(probs)[-5:][::-1]
print("\nTop 5 predictions:")
for i in top5:
    print(f"  {repr(itos[i])}: {probs[i]:.4f}")
```

```
Input sequence (decoded): THE PROJ
Target character: E
Predicted character (before training): ?

Top 5 predictions:
  '?': 0.0119
  'z': 0.0119
  'S': 0.0119
  '*': 0.0119
  'y': 0.0119
```

Cell 14: Cross-Entropy Loss Function

The loss function measures how far off the model's predictions are from the true answer.

Cross-entropy loss = `-log(predicted probability of the correct character)`

- If the model assigns probability 1.0 to the correct character → loss = 0 (perfect)
- If the model assigns probability 0.01 → loss ≈ 4.6 (very bad)

We add a tiny epsilon (1e-9) to prevent `log(0)` which would be -infinity.

In [14]:
```
def cross_entropy_loss(probs, target_id):
    """
    Compute cross-entropy loss for a single prediction.

    Args:
        probs: probability distribution over vocab (output of forward pass)
        target_id: integer ID of the correct next character

    Returns:
        Scalar loss value
    """
    return -np.log(probs[target_id] + 1e-9)
```

Cell 15: Training Loop (First Pass — Small Model)

Train the model using **stochastic gradient descent (SGD)** with manual backpropagation.

For each training example:

1. **Forward pass**: Compute predictions
2. **Compute loss**: Cross-entropy between prediction and target
3. **Backpropagation**: Compute gradients of loss w.r.t. every weight
4. **Update weights**: Nudge each weight in the direction that reduces loss

We also generate sample text periodically to see the model's progress.

In [15]:
```
# Hyperparameters
learning_rate = 0.1       # step size for gradient descent
num_epochs = 5            # number of passes over the dataset

# Training loop
for epoch in range(num_epochs):
    total_loss = 0

    # Loop over each input-target pair
    for i in range(len(x_manual)):
        x_seq = x_manual[i]    # input sequence of token IDs
        y_true = y_manual[i]   # target token ID

        # ---- Forward Pass ----
```

```
        embeds = W_embed[x_seq]                  # Lookup embeddings: block_
        h_input = embeds.flatten()               # Flatten to single vector
        h = np.tanh(h_input @ W1 + b1)           # Hidden layer with tanh ac
        logits = h @ W2 + b2                     # Output logits
        probs = softmax(logits)                  # Probabilities over vocab

        # ---- Compute Loss (Cross-Entropy) ----
        loss = cross_entropy_loss(probs, y_true)
        total_loss += loss

        # ---- Backpropagation ----
        # Gradient of loss w.r.t logits (softmax + cross-entropy combined)
        dlogits = probs.copy()
        dlogits[y_true] -= 1  # subtract 1 at the true class

        # Gradients for output layer
        dW2 = np.outer(h, dlogits)   # hidden_dim x vocab_size
        db2 = dlogits                # vocab_size

        # Gradients for hidden layer
        dh = dlogits @ W2.T                     # propagate gradient back
        dh_raw = dh * (1 - h**2)                # tanh derivative: 1 - tanh(x

        dW1 = np.outer(h_input, dh_raw)         # input_dim x hidden_dim
        db1 = dh_raw                            # hidden_dim

        # Gradients for embeddings
        dembed_flat = dh_raw @ W1.T             # block_size * embedding_dim
        dembed = dembed_flat.reshape(block_size, embedding_dim)
        dW_embed = np.zeros_like(W_embed)
        for j, idx in enumerate(x_seq):
            dW_embed[idx] += dembed[j]          # accumulate grads for repeat

        # ---- Update Weights (SGD) ----
        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2
        W1 -= learning_rate * dW1
        b1 -= learning_rate * db1
        W_embed -= learning_rate * dW_embed

    # End of epoch: print average loss and generate sample text
    avg_loss = total_loss / len(x_manual)
    print(f"Epoch {epoch+1}/{num_epochs}, Average Loss: {avg_loss:.4f}")

    # Generate a sample of 50 characters
    start_idx = np.random.randint(0, len(x_manual))
    generated_seq = x_manual[start_idx].copy()
    print("Sample start:", decode(generated_seq))

    for _ in range(50):
        probs = forward(generated_seq[-block_size:])
        next_id = np.random.choice(len(probs), p=probs)
        generated_seq.append(next_id)

    print("Generated text:", decode(generated_seq))
    print("---")
```

```
Epoch 1/5, Average Loss: 3.5672
Sample start: ler and
Generated text: ler and g ste tile;s whes anrnessany d byayhessai se verhi
---
Epoch 2/5, Average Loss: 3.5696
Sample start: Archange
Generated text: Archanged. ninhobastsin
f,ad aad . ays ndseoin was in fkd
---
Epoch 3/5, Average Loss: 3.5615
Sample start: efore my
Generated text: efore my eld esde tornott
thin-raclolosawescay andes
by Hl
---
Epoch 4/5, Average Loss: 3.5465
Sample start: f counte
Generated text: f counte nee thenkecthey bgrin end d d tianandts mant ine
---
Epoch 5/5, Average Loss: 3.5229
Sample start: rangers.
Generated text: rangers.ed ans
aassnink
lovanlenciand d anct.e lonangand.
---
```

Cell 16: Scale Up — Larger Model with Longer Context

Now we increase the model capacity:

- `block_size` : 8 → 16 (sees more context)
- `embedding_dim` : 16 → 32 (richer character representations)
- `hidden_dim` : 32 → 64 (more expressive hidden layer)

We also need to **rebuild the training data** with the new block_size, and reinitialize all weights to match the new dimensions.

In [16]:
```
# New hyperparameters
block_size = 16
embedding_dim = 32
hidden_dim = 64
vocab_size = len(chars)

# Rebuild training data with new block_size
x_manual = []
y_manual = []
for i in range(len(encoded_text) - block_size):
    x_manual.append(encoded_text[i:i+block_size])
    y_manual.append(encoded_text[i + block_size])

print(f"Training examples: {len(x_manual)} (with block_size={block_size})")

# Reinitialize weights with proper dimensions
```

```
np.random.seed(42)
W_embed = np.random.randn(vocab_size, embedding_dim) * 0.01
W1 = np.random.randn(block_size * embedding_dim, hidden_dim) * 0.01  # 16*32
b1 = np.zeros(hidden_dim)
W2 = np.random.randn(hidden_dim, vocab_size) * 0.01
b2 = np.zeros(vocab_size)

print(f"W_embed: {W_embed.shape}, W1: {W1.shape}, W2: {W2.shape}")
```

```
Training examples: 419393 (with block_size=16)
W_embed: (84, 32), W1: (512, 64), W2: (64, 84)
```

Cell 17: Text Generation Function

A reusable function to generate text from the trained model.

Key generation controls:

- **Temperature**: Controls randomness. Lower = more conservative (picks common characters), Higher = more creative/risky
- **Top-k filtering**: Only sample from the top k most likely characters, zeroing out the rest. Prevents very unlikely characters from being chosen.

In [17]:
```
def generate_text(start_seq, length=200, temperature=0.8, top_k=5):
    """Generate text from a starting sequence using the current model.

    Args:
        start_seq: list of token IDs to start from (at least block_size long
        length: number of characters to generate
        temperature: controls randomness (lower = more deterministic)
        top_k: only sample from top k most likely characters

    Returns:
        Full generated sequence (start + generated) as list of token IDs
    """
    generated = start_seq.copy()

    for _ in range(length):
        # Take the last block_size tokens as context
        context = generated[-block_size:]

        # Forward pass
        embeds = W_embed[context].flatten()
        h = np.tanh(embeds @ W1 + b1)
        logits = h @ W2 + b2
        probs = np.exp(logits) / np.sum(np.exp(logits))  # softmax

        # Apply temperature scaling (raise probs to power 1/T, then renormal
        probs = probs ** (1 / temperature)
        probs /= probs.sum()

        # Apply top-k filtering: keep only top k probabilities
        if top_k is not None:
```

```
            top_idx = np.argsort(probs)[-top_k:]
            mask = np.zeros_like(probs)
            mask[top_idx] = probs[top_idx]
            probs = mask / mask.sum()

        # Sample the next token from the probability distribution
        next_id = np.random.choice(len(probs), p=probs)
        generated.append(next_id)

    return generated
```

Cell 18: Train the Larger Model (20 Epochs)

Train the scaled-up model for 20 epochs with the same SGD + backpropagation approach. Every 5 epochs, we generate a 200-character sample to monitor quality.

Note: This will take a while since we're training on the full text with pure NumPy (no GPU). The loss should decrease over epochs, and generated text should become more English-like.

In [18]:
```
learning_rate = 0.1
num_epochs = 20

for epoch in range(num_epochs):
    total_loss = 0

    for i in range(len(x_manual)):
        x_seq = x_manual[i]
        y_true = y_manual[i]

        # ---- Forward pass ----
        embeds = W_embed[x_seq].flatten()
        h = np.tanh(embeds @ W1 + b1)
        logits = h @ W2 + b2
        probs = np.exp(logits) / np.sum(np.exp(logits))  # softmax

        # ---- Loss ----
        loss = -np.log(probs[y_true] + 1e-9)
        total_loss += loss

        # ---- Backpropagation ----
        dlogits = probs.copy()
        dlogits[y_true] -= 1

        dW2 = np.outer(h, dlogits)
        db2 = dlogits

        dh = dlogits @ W2.T
        dh_raw = dh * (1 - h**2)  # tanh derivative

        dW1 = np.outer(embeds, dh_raw)
        db1 = dh_raw
```

```
        dembed_flat = dh_raw @ W1.T
        dembed = dembed_flat.reshape(block_size, embedding_dim)
        dW_embed = np.zeros_like(W_embed)
        for j, idx in enumerate(x_seq):
            dW_embed[idx] += dembed[j]

        # ---- Update weights (SGD) ----
        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2
        W1 -= learning_rate * dW1
        b1 -= learning_rate * db1
        W_embed -= learning_rate * dW_embed

    # Print average loss for this epoch
    avg_loss = total_loss / len(x_manual)
    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {avg_loss:.4f}")

    # Generate sample text every 5 epochs
    if (epoch + 1) % 5 == 0:
        start_idx = np.random.randint(0, len(x_manual))
        start_seq = x_manual[start_idx]
        generated_seq = generate_text(start_seq, length=200, temperature=0.8
        print("Sample generation:")
        print(decode(generated_seq))
        print("---")
```

```
Epoch 1/20, Loss: 5.3722
Epoch 2/20, Loss: 5.4459
Epoch 3/20, Loss: 5.4606
Epoch 4/20, Loss: 5.4705
Epoch 5/20, Loss: 5.4701
Sample generation:
hero, whose extian thi leytay
 lT wao was ano ens ano adn bn erdtpan wn  bo  aystaosihhir
e wasem bn thb vrs ane lns bls ai waat an who te  pau ansthagiwn ahae thoe;c
nn.lgn tls aistut.t ahl ans toct.tsia. wtes bhtt.a
---
Epoch 6/20, Loss: 5.4809
Epoch 7/20, Loss: 5.4786
Epoch 8/20, Loss: 5.4636
Epoch 9/20, Loss: 5.4820
Epoch 10/20, Loss: 5.4908
Sample generation:
he productions oe thslinte.ne wass yc wh
 wa sooeth
 whstaycewe shalebhenhw.n  eonst
i weseiardnh bn
 w st nk Hre ta t st stno a sdin saio emisdectesce se m stes
 wne he sase yhsonn wd sestan tn eristiltedf
nhaoh
teo
---
Epoch 11/20, Loss: 5.4929
Epoch 12/20, Loss: 5.5137
Epoch 13/20, Loss: 5.5134
Epoch 14/20, Loss: 5.4951
Epoch 15/20, Loss: 5.4865
Sample generation:
yourself. And yesededn.ank. cokere td
cl ttstee s des tesn nsr srd uoteatlwa
r nardlnse lodnalhfy
thtnresetld He tkdt shrnselirrs
.e and. sotnse.na
 blalethbcdetoses

not dndese to
cvnt dhedaaldnwneHn sert Ihrh
noaso
---
Epoch 16/20, Loss: 5.4720
Epoch 17/20, Loss: 5.4608
Epoch 18/20, Loss: 5.4522
Epoch 19/20, Loss: 5.4582
Epoch 20/20, Loss: 5.4574
Sample generation:
ent of a nervousn
 ca t ot nhet; nn
 faitrithdt dne weota cas
 thetw nn
nc cas
n ntewe ofv
 n t d apvn,ls.nd the nestli.n tee t strnse s sna ces wlst thende nm caitpes
```

```
s.ntn thit inxmhas.tne n.
e bnasli
et seot inn
u
---
```

Cell 19: Debug — Verify Tensor Shapes

A quick sanity check to confirm all intermediate tensors have the expected shapes. This is essential for catching dimension mismatches that cause cryptic errors.

In [19]:
```
# Verify shapes through the forward pass
x_seq = x_manual[0]
embeds = W_embed[x_seq].flatten()
print("embeds.shape:", embeds.shape)      # Expected: (block_size * embeddir
print("W1.shape:", W1.shape)              # Expected: (512, 64)

h = np.tanh(embeds @ W1 + b1)
print("h.shape:", h.shape)                # Expected: (64,)

logits = h @ W2 + b2
print("logits.shape:", logits.shape)       # Expected: (vocab_size,)
print("\nAll shapes look correct!")
```

```
embeds.shape: (512,)
W1.shape: (512, 64)
h.shape: (64,)
logits.shape: (84,)

All shapes look correct!
```

Cell 20: Generate Final Sample

Use the fully trained model to generate a longer passage of text. Try different temperature and top_k values to see how they affect output quality.

In [20]:
```
# Pick a random starting context from the training data
start_idx = np.random.randint(0, len(x_manual))
start_seq = x_manual[start_idx]

print("=" * 60)
print("FINAL GENERATION (temperature=0.8, top_k=5)")
print("=" * 60)
generated = generate_text(start_seq, length=300, temperature=0.8, top_k=5)
print(decode(generated))

print("\n" + "=" * 60)
print("LOWER TEMPERATURE (temperature=0.5, top_k=5) — more conservative")
print("=" * 60)
generated = generate_text(start_seq, length=300, temperature=0.5, top_k=5)
print(decode(generated))
```

```python
print("\n" + "=" * 60)
print("HIGHER TEMPERATURE (temperature=1.2, top_k=10) — more creative")
print("=" * 60)
generated = generate_text(start_seq, length=300, temperature=1.2, top_k=10)
print(decode(generated))
```

```
============================================================
FINAL GENERATION (temperature=0.8, top_k=5)
============================================================
foe. If I were an che
de sais ileentrnt o de ekitnn
n
otr tist tce
n Unane st spidtrlirovtheest ct tai
adhcanmrna f s adsesw nn
fe masoees.a,na dae woswln d d a seawcocrn
 y se the
we nadt n.vtnse s s iedstn..ndne them;e nt chispl neot sy sle nesthi.fidae t
sisoses.so sne t stan
e.
elit theest,n s d e wesnroct s l

============================================================
LOWER TEMPERATURE (temperature=0.5, top_k=5) — more conservative
============================================================
foe. If I were an chisastn nnd cnot in,s; cfe nest nonesisn nnase hnrn boatl
runs nn t slnesecnt n
 t son nesnc cf deest snr ciete snst nct ha
nsdce.
no tee to
thitrs
icnt ncst nlthd s s sistuwcrsone e so nae.n nn
nc c
a
 aneew rn
pc teaween.rh r s wese.”n
o ne saeswese
do n
 tnese.s.rive tee.w bna nastawcrn
r naata

============================================================
HIGHER TEMPERATURE (temperature=1.2, top_k=10) — more creative
============================================================
foe. If I were a.” indttrsfn
ne sa sae
wustr .ao libtdn.e.vnanm yle chrt no whd
n nerwerot in darisesnw..nl nide th Tn dnesfrhnrd uee wnitaosib
dene wa a nkess nnrncv.hok rne wlat bkrsteserdnv,a saacetsibn bn tedt,bhys m
iee ine
n nnancicafoctht
si nyeelsus
 r itest,.!p n  maiweswsbn bhv se ceiSf”splo rhv ,pdeebs!n
```

Part 2: Upgrading to a Transformer

Our feedforward model above treats all context positions equally — it just flattens embeddings and hopes for the best. Now we build a proper **decoder-only transformer** (GPT-style) from scratch, still in pure NumPy.

What changes:

Component	Part 1 (MLP)	Part 2 (Transformer)
Architecture	Flatten → Hidden → Output	Self-Attention → FFN → Output
Context handling	All positions weighted equally	Attention dynamically weights positions
Position info	Implicit (position in flattened vector)	Explicit positional embeddings
Normalization	None	Layer Normalization
Residual connections	None	Yes
Optimizer	Vanilla SGD	Adam
Batching	Single example at a time	Mini-batches of 32

Cell 21: Additional Imports

We need `time` to track training speed. Everything else was imported in Cell 1.

In [21]:
```
import time

# We'll reuse: np, encode, decode, chars, vocab_size, stoi, itos, encoded_te
# from Part 1 — no need to redefine them.
print(f"Carrying over from Part 1: vocab_size={vocab_size}, text length={ler
```

```
Carrying over from Part 1: vocab_size=84, text length=419,409
```

Cell 22: Transformer Hyperparameters

These define the shape of our transformer. We keep things small so it trains in reasonable time on CPU.

- **block_size**: How many characters the model sees at once (context window)
- **n_embd**: Embedding dimension — the "width" of the network
- **n_head**: Number of attention heads (each looks for different patterns)
- **n_layer**: Number of stacked transformer blocks (depth)
- **head_dim**: Each head operates on `n_embd / n_head` dimensions

In [22]:
```python
# Model architecture
block_size = 32       # context window (characters the model sees)
n_embd = 64           # embedding dimension
n_head = 4            # number of attention heads
n_layer = 2           # number of transformer blocks
head_dim = n_embd // n_head  # 64 / 4 = 16 dims per head

# Training
batch_size = 32       # examples per gradient update (mini-batch)
learning_rate = 3e-4  # Adam learning rate

print(f"Context window: {block_size} characters")
print(f"Embedding dim: {n_embd}")
print(f"Attention heads: {n_head} (each {head_dim}-dim)")
print(f"Transformer layers: {n_layer}")
print(f"Batch size: {batch_size}")
```

```
Context window: 32 characters
Embedding dim: 64
Attention heads: 4 (each 16-dim)
Transformer layers: 2
Batch size: 32
```

Cell 23: Mini-Batch Training Data

Unlike Part 1 which processed one example at a time (very noisy gradients), we now use **mini-batches** — processing 32 examples simultaneously.

Also, we now predict the next token at **every** position in the sequence, not just the last one. This gives us `block_size` predictions per example — much more training signal per forward pass.

In [23]:
```python
data = np.array(encoded_text, dtype=np.int32)

def get_batch(batch_size):
    """
    Sample a random mini-batch of training examples.

    Returns:
        x: (batch_size, block_size) input token IDs
        y: (batch_size, block_size) target token IDs (shifted by 1)
    """
    ix = np.random.randint(0, len(data) - block_size - 1, size=batch_size)
    x = np.array([data[i:i+block_size] for i in ix])
    y = np.array([data[i+1:i+block_size+1] for i in ix])
    return x, y

# Test it
xb, yb = get_batch(4)
print(f"Input batch shape:  {xb.shape}")   # (4, 32)
print(f"Target batch shape: {yb.shape}")   # (4, 32)
print(f"\nExample input:  '{decode(xb[0])}'")
```

```
print(f"Example target: '{decode(yb[0])}'")
print("\nTarget is the input shifted right by 1 character.")
```

```
Input batch shape:  (4, 32)
Target batch shape: (4, 32)

Example input:  'ain went out in search of berrie'
Example target: 'in went out in search of berries'

Target is the input shifted right by 1 character.
```

Cell 24: Building Blocks — Softmax, Layer Norm, GELU

Core utility functions used throughout the transformer:

- **Softmax**: Converts raw scores to probabilities (used in attention and output)
- **Layer Normalization**: Normalizes activations to mean=0, std=1 then scales/shifts with learned parameters. This prevents values from exploding or vanishing as data flows through many layers.
- **GELU**: Activation function used in the feedforward network. Smoother than ReLU — used in GPT-2 and all modern transformers.

In [24]:
```
def softmax_transformer(x, axis=-1):
    """Numerically stable softmax along given axis."""
    e_x = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e_x / np.sum(e_x, axis=axis, keepdims=True)


def layer_norm_forward(x, gamma, beta, eps=1e-5):
    """
    Layer Normalization forward pass.

    For each token's embedding vector, normalize to mean=0 and std=1,
    then scale by learned gamma and shift by learned beta.

    Args:
        x: (..., n_embd) input
        gamma: (n_embd,) learned scale
        beta: (n_embd,) learned shift
    Returns:
        out: normalized output
        cache: values needed for backward pass
    """
    mean = x.mean(axis=-1, keepdims=True)
    var = x.var(axis=-1, keepdims=True)
    x_norm = (x - mean) / np.sqrt(var + eps)
    out = gamma * x_norm + beta
    cache = (x, x_norm, mean, var, gamma, beta, eps)
    return out, cache


def layer_norm_backward(dout, cache):
```

```python
    """Backward pass for layer normalization."""
    x, x_norm, mean, var, gamma, beta, eps = cache
    N = x.shape[-1]

    dgamma = np.sum(dout * x_norm, axis=tuple(range(x.ndim - 1)))
    dbeta = np.sum(dout, axis=tuple(range(x.ndim - 1)))

    dx_norm = dout * gamma
    std_inv = 1.0 / np.sqrt(var + eps)
    dx = (1.0 / N) * std_inv * (
        N * dx_norm
        - np.sum(dx_norm, axis=-1, keepdims=True)
        - x_norm * np.sum(dx_norm * x_norm, axis=-1, keepdims=True)
    )
    return dx, dgamma, dbeta


def gelu(x):
    """GELU activation — smoother than ReLU, used in GPT-2+."""
    return 0.5 * x * (1 + np.tanh(np.sqrt(2 / np.pi) * (x + 0.044715 * x**3)


def gelu_backward(x):
    """Derivative of GELU."""
    c = np.sqrt(2 / np.pi)
    inner = c * (x + 0.044715 * x**3)
    tanh_inner = np.tanh(inner)
    sech2 = 1 - tanh_inner**2
    dinner = c * (1 + 3 * 0.044715 * x**2)
    return 0.5 * (1 + tanh_inner) + 0.5 * x * sech2 * dinner


# Quick test
print("GELU([-1, 0, 1, 2]):", gelu(np.array([-1.0, 0.0, 1.0, 2.0])).round(3)
print("Softmax([1, 2, 3]):", softmax_transformer(np.array([1.0, 2.0, 3.0])).
print("All utility functions defined.")
```

```
GELU([-1, 0, 1, 2]): [-0.159  0.     0.841  1.955]
Softmax([1, 2, 3]): [0.09  0.245 0.665]
All utility functions defined.
```

Cell 25: Initialize Transformer Parameters

The full parameter set for our transformer:

Embeddings:

- `wte` : Token embeddings — each of 84 characters gets a 64-dim vector
- `wpe` : Position embeddings — each of 32 positions gets a 64-dim vector (this is how the model knows *where* a character is in the sequence)

Per Transformer Block:

- `W_qkv` : Projects embeddings into Query, Key, Value vectors for attention

- `W_out` : Projects attention output back to embedding dimension
- `W_fc1` : FFN expand layer (64 → 256)
- `W_fc2` : FFN compress layer (256 → 64)
- `ln1/ln2` : Layer norm parameters (gamma and beta)

Output head:

- `W_lm_head` : Maps final hidden states to vocabulary logits

In [25]:
```
np.random.seed(42)

def init_params():
    """Initialize all transformer parameters."""
    params = {}
    scale = 0.02

    # Token and position embeddings
    params['wte'] = np.random.randn(vocab_size, n_embd) * scale
    params['wpe'] = np.random.randn(block_size, n_embd) * scale

    # Transformer blocks
    for layer in range(n_layer):
        p = f'layer{layer}'

        # Layer norm 1 (before attention)
        params[f'{p}_ln1_gamma'] = np.ones(n_embd)
        params[f'{p}_ln1_beta'] = np.zeros(n_embd)

        # Attention: combined QKV projection
        params[f'{p}_W_qkv'] = np.random.randn(n_embd, 3 * n_embd) * scale
        params[f'{p}_b_qkv'] = np.zeros(3 * n_embd)

        # Attention output projection
        params[f'{p}_W_out'] = np.random.randn(n_embd, n_embd) * scale
        params[f'{p}_b_out'] = np.zeros(n_embd)

        # Layer norm 2 (before FFN)
        params[f'{p}_ln2_gamma'] = np.ones(n_embd)
        params[f'{p}_ln2_beta'] = np.zeros(n_embd)

        # Feedforward: expand then compress
        params[f'{p}_W_fc1'] = np.random.randn(n_embd, 4 * n_embd) * scale
        params[f'{p}_b_fc1'] = np.zeros(4 * n_embd)
        params[f'{p}_W_fc2'] = np.random.randn(4 * n_embd, n_embd) * scale
        params[f'{p}_b_fc2'] = np.zeros(n_embd)

    # Final layer norm
    params['ln_f_gamma'] = np.ones(n_embd)
    params['ln_f_beta'] = np.zeros(n_embd)

    # Output projection (language model head)
    params['W_lm_head'] = np.random.randn(n_embd, vocab_size) * scale
    params['b_lm_head'] = np.zeros(vocab_size)
```

```
    return params


params = init_params()
total_params = sum(p.size for p in params.values())
print(f"Total parameters: {total_params:,}")
print(f"\nParameter breakdown:")
for name, p in params.items():
    print(f"  {name:30s} {str(p.shape):20s} {p.size:>8,}")
```

```
Total parameters: 112,980

Parameter breakdown:
  wte                            (84, 64)                5,376
  wpe                            (32, 64)                2,048
  layer0_ln1_gamma               (64,)                      64
  layer0_ln1_beta                (64,)                      64
  layer0_W_qkv                   (64, 192)              12,288
  layer0_b_qkv                   (192,)                    192
  layer0_W_out                   (64, 64)                4,096
  layer0_b_out                   (64,)                      64
  layer0_ln2_gamma               (64,)                      64
  layer0_ln2_beta                (64,)                      64
  layer0_W_fc1                   (64, 256)              16,384
  layer0_b_fc1                   (256,)                    256
  layer0_W_fc2                   (256, 64)              16,384
  layer0_b_fc2                   (64,)                      64
  layer1_ln1_gamma               (64,)                      64
  layer1_ln1_beta                (64,)                      64
  layer1_W_qkv                   (64, 192)              12,288
  layer1_b_qkv                   (192,)                    192
  layer1_W_out                   (64, 64)                4,096
  layer1_b_out                   (64,)                      64
  layer1_ln2_gamma               (64,)                      64
  layer1_ln2_beta                (64,)                      64
  layer1_W_fc1                   (64, 256)              16,384
  layer1_b_fc1                   (256,)                    256
  layer1_W_fc2                   (256, 64)              16,384
  layer1_b_fc2                   (64,)                      64
  ln_f_gamma                     (64,)                      64
  ln_f_beta                      (64,)                      64
  W_lm_head                      (64, 84)                5,376
  b_lm_head                      (84,)                      84
```

Cell 26: Causal Self-Attention — Forward Pass

This is the **core innovation** of the transformer:

1. Each token produces **Query (Q)**, **Key (K)**, **Value (V)** vectors:
   - Q = "What am I looking for?"
   - K = "What do I contain?"
   - V = "What information do I pass along?"
2. Attention score = Q · K^T / √(head_dim) — how relevant is each other token?

3. **Causal mask**: Block future positions so the model can't cheat by peeking ahead.

4. Softmax the scores into weights, then compute weighted sum of V.

Multi-head: We split into `n_head` independent attention computations, each looking for different patterns, then concatenate the results.

In [26]:
```
# Pre-compute causal mask: lower-triangular matrix
# Token i can only attend to tokens 0..i (not future tokens)
causal_mask = np.tril(np.ones((block_size, block_size), dtype=bool))

print("Causal mask (first 8x8):")
print(causal_mask[:8, :8].astype(int))
print("\nToken 0 sees only itself. Token 7 sees positions 0-7.")


def attention_forward(x, W_qkv, b_qkv, W_out, b_out):
    """
    Multi-head causal self-attention.

    Args:
        x: (B, T, C) input embeddings
    Returns:
        out: (B, T, C) attention output
        cache: values saved for backprop
    """
    B, T, C = x.shape

    # Project to Q, K, V (combined for efficiency)
    qkv = x @ W_qkv + b_qkv         # (B, T, 3*C)
    q, k, v = np.split(qkv, 3, axis=-1)  # each (B, T, C)

    # Reshape for multi-head: (B, T, C) -> (B, n_head, T, head_dim)
    q = q.reshape(B, T, n_head, head_dim).transpose(0, 2, 1, 3)
    k = k.reshape(B, T, n_head, head_dim).transpose(0, 2, 1, 3)
    v = v.reshape(B, T, n_head, head_dim).transpose(0, 2, 1, 3)

    # Attention scores: Q @ K^T / sqrt(head_dim)
    scale = 1.0 / np.sqrt(head_dim)
    att_scores = (q @ k.transpose(0, 1, 3, 2)) * scale  # (B, n_head, T, T)

    # Apply causal mask (future positions -> -inf -> softmax gives 0)
    mask = causal_mask[:T, :T]
    att_scores = np.where(mask, att_scores, -1e9)

    # Softmax -> attention weights
    att_weights = softmax_transformer(att_scores, axis=-1)  # (B, n_head, T,

    # Weighted combination of values
    att_output = att_weights @ v  # (B, n_head, T, head_dim)

    # Concatenate heads: (B, n_head, T, head_dim) -> (B, T, C)
    att_output = att_output.transpose(0, 2, 1, 3).reshape(B, T, C)
```

```
    # Output projection
    out = att_output @ W_out + b_out

    cache = (x, q, k, v, att_weights, att_output, W_qkv, b_qkv, W_out, b_out
    return out, cache


print("\nAttention forward pass defined.")
```

```
Causal mask (first 8x8):
[[1 0 0 0 0 0 0 0]
 [1 1 0 0 0 0 0 0]
 [1 1 1 0 0 0 0 0]
 [1 1 1 1 0 0 0 0]
 [1 1 1 1 1 0 0 0]
 [1 1 1 1 1 1 0 0]
 [1 1 1 1 1 1 1 0]
 [1 1 1 1 1 1 1 1]]

Token 0 sees only itself. Token 7 sees positions 0-7.

Attention forward pass defined.
```

Cell 27: Attention Backward Pass

Backpropagation through multi-head attention. We reverse every operation from the forward pass, propagating gradients from the output back through the softmax, the Q/K/V projections, and the multi-head reshape.

In [27]:

```
def attention_backward(dout, cache):
    """
    Backward pass through multi-head causal self-attention.

    Args:
        dout: (B, T, C) gradient from the next layer
        cache: saved values from forward pass
    Returns:
        dx: gradient w.r.t. input
        grads: dict of parameter gradients
    """
    x, q, k, v, att_weights, att_output, W_qkv, b_qkv, W_out, b_out = cache
    B, T, C = x.shape

    # Backward through output projection
    dW_out = att_output.reshape(-1, C).T @ dout.reshape(-1, C)
    db_out = dout.sum(axis=(0, 1))
    datt_output = dout @ W_out.T

    # Reshape to multi-head format
    datt_output = datt_output.reshape(B, T, n_head, head_dim).transpose(0, 2

    # Backward through att_weights @ v
    datt_weights = datt_output @ v.transpose(0, 1, 3, 2)
    dv = att_weights.transpose(0, 1, 3, 2) @ datt_output
```

```
    # Backward through softmax
    datt_scores = att_weights * (datt_weights - (att_weights * datt_weights)

    # Apply causal mask to gradient
    mask = causal_mask[:T, :T]
    datt_scores = np.where(mask, datt_scores, 0.0)

    # Backward through scaling
    datt_scores *= 1.0 / np.sqrt(head_dim)

    # Backward through Q @ K^T
    dq = datt_scores @ k
    dk = datt_scores.transpose(0, 1, 3, 2) @ q

    # Reshape heads back to (B, T, C)
    dq = dq.transpose(0, 2, 1, 3).reshape(B, T, C)
    dk = dk.transpose(0, 2, 1, 3).reshape(B, T, C)
    dv = dv.transpose(0, 2, 1, 3).reshape(B, T, C)

    # Concatenate Q, K, V gradients and backprop through projection
    dqkv = np.concatenate([dq, dk, dv], axis=-1)
    dW_qkv = x.reshape(-1, C).T @ dqkv.reshape(-1, 3 * C)
    db_qkv = dqkv.sum(axis=(0, 1))
    dx = dqkv @ W_qkv.T

    grads = {'W_qkv': dW_qkv, 'b_qkv': db_qkv, 'W_out': dW_out, 'b_out': db_
    return dx, grads


print("Attention backward pass defined.")
```

```
Attention backward pass defined.
```

Cell 28: Feedforward Network (Forward + Backward)

After attention gathers relevant context, the FFN processes it:

1. **Expand**: 64 → 256 (4× expansion gives the network room to compute)
2. **GELU activation**: Non-linear transformation
3. **Compress**: 256 → 64 (back to original size)

Think of attention as "gathering info" and FFN as "processing that info."

In [28]:
```
def ffn_forward(x, W_fc1, b_fc1, W_fc2, b_fc2):
    """Feedforward network: expand -> GELU -> compress."""
    h = x @ W_fc1 + b_fc1        # (B, T, 4*C)
    h_act = gelu(h)               # (B, T, 4*C)
    out = h_act @ W_fc2 + b_fc2   # (B, T, C)
    cache = (x, h, h_act, W_fc1, b_fc1, W_fc2, b_fc2)
    return out, cache


def ffn_backward(dout, cache):
```

```
    """Backward pass through FFN."""
    x, h, h_act, W_fc1, b_fc1, W_fc2, b_fc2 = cache
    B, T, C = x.shape
    C4 = 4 * C

    # Backward through compress layer
    dW_fc2 = h_act.reshape(-1, C4).T @ dout.reshape(-1, C)
    db_fc2 = dout.sum(axis=(0, 1))
    dh_act = dout @ W_fc2.T

    # Backward through GELU
    dh = dh_act * gelu_backward(h)

    # Backward through expand layer
    dW_fc1 = x.reshape(-1, C).T @ dh.reshape(-1, C4)
    db_fc1 = dh.sum(axis=(0, 1))
    dx = dh @ W_fc1.T

    grads = {'W_fc1': dW_fc1, 'b_fc1': db_fc1, 'W_fc2': dW_fc2, 'b_fc2': db_
    return dx, grads


print("Feedforward network (forward + backward) defined.")
```

```
Feedforward network (forward + backward) defined.
```

Cell 29: Full Transformer Forward Pass

Assemble all pieces into the complete transformer:

```
Token IDs
    ↓
Token Embedding + Position Embedding
    ↓
┌─── Transformer Block (×2) ──────────────────────┐
│  Layer Norm → Self-Attention → Add (residual)   │
│  Layer Norm → FFN → Add (residual)              │
└─────────────────────────────────────────────────┘
    ↓
Final Layer Norm → Linear → Logits
```

Residual connections (the "Add" steps) add the layer's input to its output. This lets gradients flow cleanly backward without degrading — essential for training deep networks.

In [29]:
```
def transformer_forward(x_ids, params):
    """
    Full transformer forward pass.

    Args:
        x_ids: (B, T) integer token IDs
        params: dict of all model parameters
```

```
    Returns:
        logits: (B, T, vocab_size) prediction scores
        cache: everything needed for backprop
    """
    B, T = x_ids.shape
    all_caches = {}

    # --- Embeddings ---
    tok_emb = params['wte'][x_ids]    # (B, T, n_embd)
    pos_emb = params['wpe'][:T]        # (T, n_embd)
    x = tok_emb + pos_emb              # combine token + position info
    all_caches['emb_input'] = x_ids

    # --- Transformer Blocks ---
    for layer in range(n_layer):
        p = f'layer{layer}'

        # --- Sub-block 1: Attention ---
        residual1 = x
        x_ln1, ln1_cache = layer_norm_forward(x, params[f'{p}_ln1_gamma'], p
        all_caches[f'{p}_ln1'] = ln1_cache

        att_out, att_cache = attention_forward(
            x_ln1, params[f'{p}_W_qkv'], params[f'{p}_b_qkv'],
            params[f'{p}_W_out'], params[f'{p}_b_out'])
        all_caches[f'{p}_att'] = att_cache

        x = residual1 + att_out  # residual connection
        all_caches[f'{p}_residual1'] = residual1

        # --- Sub-block 2: FFN ---
        residual2 = x
        x_ln2, ln2_cache = layer_norm_forward(x, params[f'{p}_ln2_gamma'], p
        all_caches[f'{p}_ln2'] = ln2_cache

        ffn_out, ffn_cache = ffn_forward(
            x_ln2, params[f'{p}_W_fc1'], params[f'{p}_b_fc1'],
            params[f'{p}_W_fc2'], params[f'{p}_b_fc2'])
        all_caches[f'{p}_ffn'] = ffn_cache

        x = residual2 + ffn_out  # residual connection
        all_caches[f'{p}_residual2'] = residual2

    # --- Final Layer Norm + Output ---
    x_ln, lnf_cache = layer_norm_forward(x, params['ln_f_gamma'], params['lr
    all_caches['ln_f'] = lnf_cache

    logits = x_ln @ params['W_lm_head'] + params['b_lm_head']
    all_caches['final_hidden'] = x_ln

    return logits, all_caches


# Test it
xb, yb = get_batch(4)
logits, caches = transformer_forward(xb, params)
```

```
print(f"Input shape:  {xb.shape}")       # (4, 32)
print(f"Logits shape: {logits.shape}")    # (4, 32, 84)
print(f"→ {vocab_size} scores for each of {block_size} positions, across {4}
```

```
Input shape:  (4, 32)
Logits shape: (4, 32, 84)
→ 84 scores for each of 32 positions, across 4 batch examples.
```

Cell 30: Loss Computation + Full Backward Pass

Compute cross-entropy loss across all positions in the batch, then backpropagate through the entire transformer — from output logits all the way back to embeddings.

This is the most complex function. It reverses every operation from the forward pass.

In [30]:
```
def compute_loss_and_backward(logits, targets, caches, params):
    """
    Compute loss and backpropagate through the entire transformer.

    Returns:
        loss: scalar cross-entropy loss
        grads: dict of gradients for all parameters
    """
    B, T, V = logits.shape
    grads = {}

    # --- Loss ---
    probs = softmax_transformer(logits, axis=-1)
    log_probs = -np.log(probs[np.arange(B)[:, None], np.arange(T), targets]
    loss = log_probs.mean()

    # --- Gradient of loss w.r.t. logits ---
    dlogits = probs.copy()
    dlogits[np.arange(B)[:, None], np.arange(T), targets] -= 1
    dlogits /= (B * T)

    # --- Backward through output projection ---
    x_ln = caches['final_hidden']
    grads['W_lm_head'] = x_ln.reshape(-1, n_embd).T @ dlogits.reshape(-1, V)
    grads['b_lm_head'] = dlogits.sum(axis=(0, 1))
    dx = dlogits @ params['W_lm_head'].T

    # --- Backward through final layer norm ---
    dx, grads['ln_f_gamma'], grads['ln_f_beta'] = layer_norm_backward(dx, ca

    # --- Backward through transformer blocks (reverse order) ---
    for layer in reversed(range(n_layer)):
        p = f'layer{layer}'

        # Backward through FFN + residual
        dx_ffn = dx
        dx_ln2, ffn_grads = ffn_backward(dx_ffn, caches[f'{p}_ffn'])
        for k, v in ffn_grads.items():
            grads[f'{p}_{k}'] = v
```

```
        dx_ln2, grads[f'{p}_ln2_gamma'], grads[f'{p}_ln2_beta'] = \
            layer_norm_backward(dx_ln2, caches[f'{p}_ln2'])
        dx = dx + dx_ln2  # residual gradient

        # Backward through attention + residual
        dx_att = dx
        dx_ln1, att_grads = attention_backward(dx_att, caches[f'{p}_att'])
        for k, v in att_grads.items():
            grads[f'{p}_{k}'] = v
        dx_ln1, grads[f'{p}_ln1_gamma'], grads[f'{p}_ln1_beta'] = \
            layer_norm_backward(dx_ln1, caches[f'{p}_ln1'])
        dx = dx + dx_ln1  # residual gradient

    # --- Backward through embeddings ---
    x_ids = caches['emb_input']
    grads['wte'] = np.zeros_like(params['wte'])
    np.add.at(grads['wte'], x_ids, dx)
    grads['wpe'] = np.zeros_like(params['wpe'])
    grads['wpe'][:T] = dx.sum(axis=0)

    return loss, grads


# Test
xb, yb = get_batch(4)
logits, caches = transformer_forward(xb, params)
loss, grads = compute_loss_and_backward(logits, yb, caches, params)
print(f"Initial loss: {loss:.4f}")
print(f"Expected random loss: {-np.log(1/vocab_size):.4f} (-log(1/{vocab_siz
print(f"Gradients computed for {len(grads)} parameter tensors.")
```

```
Initial loss: 4.4280
Expected random loss: 4.4308 (-log(1/84))
Gradients computed for 30 parameter tensors.
```

Cell 31: Adam Optimizer

Part 1 used vanilla SGD (`weight -= lr * gradient`). That's noisy and slow.

Adam is much better — it maintains two running averages per weight:

- **m** (momentum): Smoothed gradient direction — prevents zig-zagging
- **v** (velocity): Smoothed squared gradient — gives each weight its own adaptive learning rate

Nearly every modern neural network uses Adam or a variant.

In [31]:
```
class AdamOptimizer:
    """Adam optimizer with per-parameter adaptive learning rates."""

    def __init__(self, params, lr=3e-4, beta1=0.9, beta2=0.999, eps=1e-8):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
```

```
        self.eps = eps
        self.t = 0
        self.m = {k: np.zeros_like(v) for k, v in params.items()}
        self.v = {k: np.zeros_like(v) for k, v in params.items()}

    def step(self, params, grads):
        """Update parameters using Adam."""
        self.t += 1
        for key in params:
            if key not in grads:
                continue
            g = grads[key]
            # Update momentum (smoothed gradient)
            self.m[key] = self.beta1 * self.m[key] + (1 - self.beta1) * g
            # Update velocity (smoothed squared gradient)
            self.v[key] = self.beta2 * self.v[key] + (1 - self.beta2) * g**2
            # Bias correction
            m_hat = self.m[key] / (1 - self.beta1**self.t)
            v_hat = self.v[key] / (1 - self.beta2**self.t)
            # Update
            params[key] -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
        return params


optimizer = AdamOptimizer(params, lr=learning_rate)
print(f"Adam optimizer initialized (lr={learning_rate})")
print(f"Tracking momentum + velocity for {len(optimizer.m)} parameter tensor
```

```
Adam optimizer initialized (lr=0.0003)
Tracking momentum + velocity for 30 parameter tensors.
```

Cell 32: Transformer Text Generation

Same idea as Part 1 — predict one character at a time — but now using the full transformer. Temperature and top-k sampling still control randomness.

In [32]:
```
def generate(params, prompt, max_new_tokens=200, temperature=0.8, top_k=10):
    """
    Generate text from the transformer.

    Args:
        params: model parameters
        prompt: starting text string
        max_new_tokens: characters to generate
        temperature: randomness (lower = more conservative)
        top_k: only sample from top k characters
    """
    tokens = encode(prompt)

    for _ in range(max_new_tokens):
        # Crop to context window
        context = tokens[-block_size:]

        # Forward pass
```

```
        x = np.array([context])
        logits, _ = transformer_forward(x, params)
        logits_last = logits[0, -1, :] / temperature

        # Softmax
        probs = softmax_transformer(logits_last)

        # Top-k filtering
        if top_k is not None:
            top_idx = np.argsort(probs)[-top_k:]
            mask = np.zeros_like(probs)
            mask[top_idx] = probs[top_idx]
            probs = mask / mask.sum()

        next_token = np.random.choice(len(probs), p=probs)
        tokens.append(next_token)

    return decode(tokens)


# Test before training
print("=" * 50)
print("GENERATION BEFORE TRAINING (random weights)")
print("=" * 50)
print(generate(params, "The creature ", max_new_tokens=100))
```

```
==================================================
GENERATION BEFORE TRAINING (random weights)
==================================================
The creature DyYvélayl-OT,EHrEj‘LDN6(-3Hp7(,U4fy DN8(o(8DNR0VR9
é
wC


y;6fD9
py wCG(,”72o
(owGG8p;
ND9N7GVV;R..(
```

Cell 33: Training Loop

Train the transformer with mini-batch SGD + Adam.

Each step: sample batch → forward pass → compute loss → backprop → Adam update.

We use **steps** rather than epochs because each step processes `batch_size × block_size` tokens randomly sampled from the text.

Note: This is pure NumPy on CPU, so it will be slow. Adjust `num_steps` based on your patience. Even 1000-2000 steps should show clear improvement.

In [33]:
```
num_steps = 3000         # total training steps (increase for better results
eval_interval = 500      # print loss every N steps
generate_interval = 1000 # generate sample every N steps
```

```
print(f"Training for {num_steps} steps")
print(f"Each step processes {batch_size} × {block_size} = {batch_size * bloc
print(f"Steps per epoch: ~{len(data) // (batch_size * block_size):,}")
print("=" * 60)

losses = []
start_time = time.time()

for step in range(num_steps):
    # Sample mini-batch
    xb, yb = get_batch(batch_size)

    # Forward pass
    logits, caches = transformer_forward(xb, params)

    # Compute loss and gradients
    loss, grads = compute_loss_and_backward(logits, yb, caches, params)

    # Update weights with Adam
    params = optimizer.step(params, grads)

    losses.append(loss)

    # Print progress
    if (step + 1) % eval_interval == 0:
        avg_loss = np.mean(losses[-eval_interval:])
        elapsed = time.time() - start_time
        sps = (step + 1) / elapsed
        print(f"Step {step+1:5d}/{num_steps} | Loss: {avg_loss:.4f} | "
              f"{sps:.1f} steps/sec | {elapsed:.0f}s elapsed")

    # Generate sample
    if (step + 1) % generate_interval == 0:
        print(f"\n--- Sample at step {step+1} ---")
        print(generate(params, "The ", max_new_tokens=150, temperature=0.8))
        print("---\n")

total_time = time.time() - start_time
print(f"\nTraining complete! {total_time:.0f}s total")
print(f"Final avg loss (last 100 steps): {np.mean(losses[-100:]):.4f}")
```

```
Training for 3000 steps
Each step processes 32 × 32 = 1024 tokens
Steps per epoch: ~409
============================================================
Step   500/3000 | Loss: 2.7025 | 15.0 steps/sec | 33s elapsed
Step  1000/3000 | Loss: 2.1926 | 14.8 steps/sec | 67s elapsed

--- Sample at step 1000 ---
The of the cespertion, the che my thid che the bears wer ive with
the the sprere the wids willl to me thin, was a wa the the bof she of thas o
and affer th
---

Step  1500/3000 | Loss: 2.0129 | 14.8 steps/sec | 101s elapsed
Step  2000/3000 | Loss: 1.9035 | 12.1 steps/sec | 165s elapsed

--- Sample at step 2000 ---
The prection a that whe pas the
ase to sthe was was the care he mow fard in
that when ther with wore more in of the cantere and the suct I she at brest
th
---

Step  2500/3000 | Loss: 1.8277 | 12.5 steps/sec | 200s elapsed
Step  3000/3000 | Loss: 1.7645 | 5.4 steps/sec | 551s elapsed

--- Sample at step 3000 ---
The time, and my and ceased firtung to here
exclect of the thun in had firstoratin tall and was she he spront a she houl
d but spiting and oble
succove to
---


Training complete! 551s total
Final avg loss (last 100 steps): 1.7482
```

Cell 34: Plot Training Loss

Visualize the loss curve. A smooth downward trend means the model is learning.

In [34]:
```
try:
    import matplotlib.pyplot as plt

    window = min(100, len(losses) // 5)
    smoothed = np.convolve(losses, np.ones(window)/window, mode='valid')

    plt.figure(figsize=(10, 4))
    plt.plot(smoothed, linewidth=0.8)
    plt.xlabel('Training Step')
    plt.ylabel('Cross-Entropy Loss')
    plt.title('Transformer Training Loss')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
```

```
    plt.show()
except ImportError:
    print("matplotlib not available. Loss summary:")
    for i in range(0, len(losses), max(1, len(losses)//10)):
        chunk = losses[max(0,i-50):i+50]
        if chunk:
            print(f"  Step {i:5d}: avg loss = {np.mean(chunk):.4f}")
```

Cell 35: Final Text Generation

Generate longer samples from the trained transformer with different prompts.

In [35]:
```
prompts = [
    "The creature ",
    "I could not ",
    "My dear friend",
    "The night was ",
]

for prompt in prompts:
    print("=" * 60)
    print(f"Prompt: '{prompt}'")
    print("=" * 60)
    print(generate(params, prompt, max_new_tokens=200, temperature=0.8, top_
    print()
```

```
============================================================
Prompt: 'The creature '
============================================================
The creature the sumount as wated the aroutined beand the souch of there
that withed the atter then watchin it
deass of the carred the to did to me made. The rister was
to to which words this and
are to deterped t

============================================================
Prompt: 'I could not '
============================================================
I could not stot as agone on to a decipessed was sirable to dear age a deat
for
the acquits the hous and worn. The was these to that
and wind the been he confice. He sumposit in the facke
and my to the
appection.

============================================================
Prompt: 'My dear friend'
============================================================
My dear friend
was of the mighter, whom the is suppeceive to tho seat a her hous to agratio
n the been the my finkss mustred mand him tark be the befort the surring to
all then mise and with the age a housed was wer

============================================================
Prompt: 'The night was '
============================================================
The night was stered, and folly
to ase to deate in to chirder to that satesid on anted to the main
clotted to the was the
ark
wo me in was of my ceanimal of those mostert. I had reathed the pears of on
are his were
```

Cell 36: Compare Generation Settings

Same prompt, different temperature and top-k to see their effect on output.

In [36]:

```python
prompt = "I felt a sensation"

settings = [
    ("Conservative", 0.5, 5),
    ("Balanced",     0.8, 10),
    ("Creative",     1.2, 20),
]

for label, temp, k in settings:
    print(f"\n{'='*60}")
    print(f"{label} (temperature={temp}, top_k={k})")
```

```
    print(f"{'='*60}")
    print(generate(params, prompt, max_new_tokens=200, temperature=temp, top
```

```
============================================================
Conservative (temperature=0.5, top_k=5)
============================================================
I felt a sensation of the sounte all was the was one my and the have was and
my had feeling to the are was and with the so the seat wind to the tee accur
ded to with hat sterely and and happess to me of the montinure wa

============================================================
Balanced (temperature=0.8, top_k=10)
============================================================
I felt a sensation. I then bours and, as in to the cottide of to these of my
felt he conting my some that tempted to me my for almany of to the
ammented all be in one appecas, my been his sward on the
wrested to me his

============================================================
Creative (temperature=1.2, top_k=20)
============================================================
I felt a sensation, and in
youis id than
dook satcerfulsh herough firibled no seath, forwards to mide
upon exclive hechsed.h.”.
But I
passedliefy ir, pigh.”

“Soon; theich hadd
oroth. Perth. Slets; yet rabing nows ow
l
```

Cell 37: Visualize Attention Weights

Peek inside the transformer to see what each attention head is looking at. This is one of the key advantages of transformers — we can interpret *what* the model learned to focus on.

In [37]:

```
# Run forward pass on specific input
test_text = "the monster walked into"
test_ids = np.array([encode(test_text)])
logits, caches = transformer_forward(test_ids, params)

# Extract attention weights from layer 0
att_weights = caches['layer0_att'][4]  # (1, n_head, T, T)
T = len(test_text)
chars_list = list(test_text)

print(f"Input: '{test_text}'")
print(f"\nWhat does the last character ('{chars_list[-1]}') attend to?\n")

for head in range(n_head):
    weights = att_weights[0, head, -1, :T]
    top5_idx = np.argsort(weights)[-5:][::-1]
```

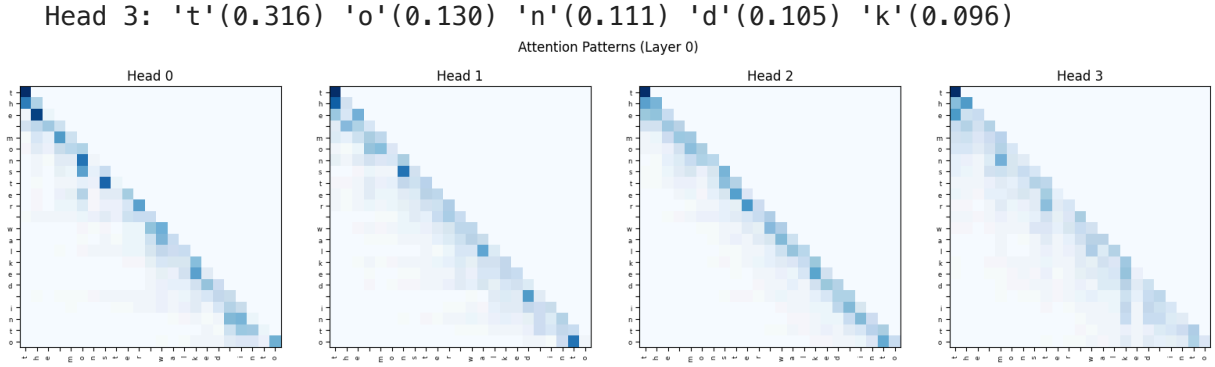
```
    print(f"  Head {head}: ", end="")
    for idx in top5_idx:
        print(f"'{chars_list[idx]}'({weights[idx]:.3f}) ", end="")
    print()

# Heatmap if matplotlib available
try:
    import matplotlib.pyplot as plt
    fig, axes = plt.subplots(1, n_head, figsize=(4 * n_head, 4))
    for head in range(n_head):
        ax = axes[head] if n_head > 1 else axes
        w = att_weights[0, head, :T, :T]
        ax.imshow(w, cmap='Blues', vmin=0)
        ax.set_title(f'Head {head}')
        ax.set_xticks(range(T))
        ax.set_yticks(range(T))
        ax.set_xticklabels(chars_list, fontsize=6, rotation=90)
        ax.set_yticklabels(chars_list, fontsize=6)
    plt.suptitle('Attention Patterns (Layer 0)', y=1.02)
    plt.tight_layout()
    plt.show()
except ImportError:
    print("\n(Install matplotlib for attention heatmaps)")
```

```
Input: 'the monster walked into'

What does the last character ('o') attend to?

  Head 0: 'o'(0.488) 'n'(0.130) 'i'(0.128) ' '(0.113) 't'(0.056)
  Head 1: 't'(0.728) 'n'(0.207) 'o'(0.032) 'd'(0.013) ' '(0.013)
  Head 2: 't'(0.519) 'o'(0.230) 'n'(0.127) 'i'(0.047) ' '(0.026)
  Head 3: 't'(0.316) 'o'(0.130) 'n'(0.111) 'd'(0.105) 'k'(0.096)
```



Cell 38: Architecture Summary

A final overview comparing what we built in Part 1 vs Part 2.

In [38]:
```
print("=" * 60)
print(" COMPLETE PROJECT SUMMARY")
print("=" * 60)
print(f"""
Dataset: Frankenstein by Mary Shelley ({len(encoded_text):,} characters, {vo

PART 1 — Feedforward Model (MLP)
  Architecture:  Flatten → Hidden(tanh) → Output(softmax)
```

```
  Context:       16 characters
  Parameters:    ~40,000
  Optimizer:     Vanilla SGD (lr=0.1)
  Key limitation: All positions weighted equally, no attention

PART 2 — Transformer (GPT-style)
  Architecture:  Token+Pos Embed → [{n_layer}× (LayerNorm → Attention → FFN)
  Context:       {block_size} characters
  Embedding dim: {n_embd}
  Heads:         {n_head} (each {head_dim}-dim)
  Layers:        {n_layer}
  Parameters:    {total_params:,}
  Optimizer:     Adam (lr={learning_rate})
  Key upgrade:   Self-attention dynamically weights context positions

Everything built from scratch in NumPy:
  ✓ Character tokenizer        ✓ Multi-head causal self-attention
  ✓ Embeddings (token + pos)    ✓ Layer normalization
  ✓ GELU activation             ✓ Residual connections
  ✓ Feedforward network          ✓ Full backpropagation
  ✓ Cross-entropy loss           ✓ Adam optimizer
  ✓ Temperature + top-k sampling ✓ Attention visualization

NOT used: PyTorch, TensorFlow, autograd, GPU, pre-trained weights
""")
print("=" * 60)
```

```
============================================================
 COMPLETE PROJECT SUMMARY
============================================================

Dataset: Frankenstein by Mary Shelley (419,409 characters, 84 unique)

PART 1 — Feedforward Model (MLP)
  Architecture:  Flatten → Hidden(tanh) → Output(softmax)
  Context:       16 characters
  Parameters:    ~40,000
  Optimizer:     Vanilla SGD (lr=0.1)
  Key limitation: All positions weighted equally, no attention

PART 2 — Transformer (GPT-style)
  Architecture:  Token+Pos Embed → [2× (LayerNorm → Attention → FFN)] → Outp
ut
  Context:       32 characters
  Embedding dim: 64
  Heads:         4 (each 16-dim)
  Layers:        2
  Parameters:    112,980
  Optimizer:     Adam (lr=0.0003)
  Key upgrade:   Self-attention dynamically weights context positions

Everything built from scratch in NumPy:
  ✓ Character tokenizer        ✓ Multi-head causal self-attention
  ✓ Embeddings (token + pos)    ✓ Layer normalization
  ✓ GELU activation             ✓ Residual connections
  ✓ Feedforward network          ✓ Full backpropagation
  ✓ Cross-entropy loss           ✓ Adam optimizer
  ✓ Temperature + top-k sampling ✓ Attention visualization

NOT used: PyTorch, TensorFlow, autograd, GPU, pre-trained weights

============================================================
```